



**RV College of  
Engineering®**

Mysore Road, RV Vidyaniketan f  
Bengaluru - 560059, Karnataka,

*Go, change the  
world*

# Machine Learning (MLOps) Tutorial: Skin disease detection system

Team No: 7

**USN**

1RV23AI088

1RV23AI093

1RV23AI091

1RV23AI090

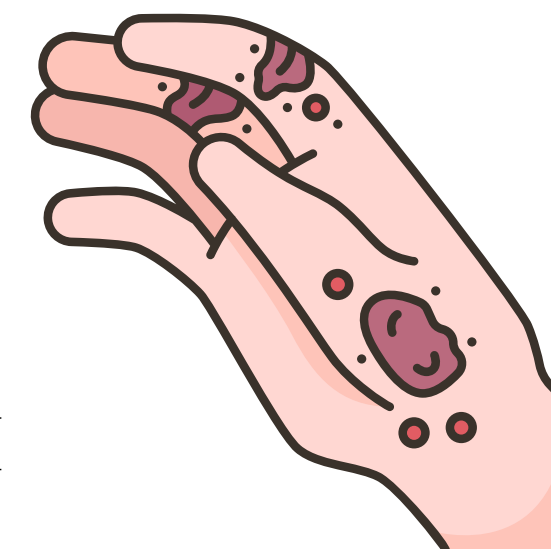
**Name**

SHALINI P

SHRAVYAA S

SHASHANK KRISHNAMANI

SHANTHESH A S



# Introduction

- Skin diseases such as Eczema, Atopic Dermatitis, Fungal Infections, Benign Keratosis-like Lesions, and Seborrheic Keratoses are common and affect people of all age groups.
- Many skin conditions exhibit **similar visual characteristics**, making accurate diagnosis challenging even for trained professionals.
- **Delayed or incorrect diagnosis** can lead to worsening of the condition, **improper treatment**, and increased healthcare costs.
- With advancements in Artificial Intelligence and Deep Learning, **image-based disease classification** has become a promising solution in healthcare.

## Motivation

- **Lack** of dermatology specialists in **rural and underserved areas**
- Manual diagnosis is time-consuming and subjective
- Need for an automated, accurate, and scalable diagnostic support system
- To leverage CNNs and MLOps practices for building a reliable and deployable skin disease detection system





# Problem Definition

To develop a deep learning-based system for accurately classifying common skin diseases such as Eczema, Atopic Dermatitis, Benign Keratosis-like Lesions, Tinea (Fungal Infections), and Seborrheic Keratoses, with the goal of reducing misdiagnosis and enabling early, effective treatment.



# Objective s

- 1 To support **early detection and diagnosis of skin conditions** to reduce the risk of incorrect or delayed treatment.
- 2 To improve **diagnostic reliability** by minimizing human subjectivity in skin disease identification.
- 3 To promote trust and transparency in **AI-based medical systems** through interpretable predictions.
- 4 To design a solution that is **scalable, maintainable, and suitable** for real-world deployment.



# Data set Description

- A publicly available skin disease image dataset is used for this project.
- The dataset consists of clinical skin images collected from dermatology sources.
- Five skin disease categories were selected based on prevalence and visual distinguishability:
  - a. Eczema
  - b. Atopic Dermatitis
  - c. Benign Keratosis-like Lesions (BKL)
  - d. Tinea (Fungal Infections)
  - e. Seborrheic Keratoses
- To avoid class imbalance, 1,250 images per class were selected, resulting in a balanced dataset.
- All images were resized to  $224 \times 224$  pixels, which is suitable for CNN-based models.





**RV College of  
Engineering®**  
Mysore Road, RV Vidyaniketan f  
Bengaluru - 560059, Karnataka,

*Go, change the  
world*

# Data set

## Description

### Five skin disease categories



Eczema



Tinea (Fungal Infections)



Atopic Dermatitis



Benign Keratosis-like  
Lesions (BKL)



Seborrheic Keratoses



# Risk Analysis

## Data-Related Risks

- Bias in skin tone representation
- Missing or mislabeled data
- Data drift in production

## Model-Related Risks

- Poor generalization to unseen data
- High false positive/negative rates
- Versioning conflicts in production

## Pipeline & Deployment Risks

- Pipeline failure due to dependency issues
- Slow inference speed affecting user experience
- Security vulnerabilities in API endpoints

## Compliance & Ethical Risks

- Lack of regulatory compliance (medical data)
- Ethical concerns around AI diagnosis
- Transparency and explainability gaps



# Risk Analysis

| Likelihood ↓ / Impact → | Negligible Risk | Low Risk            | High Risk              | Severe Risk               |
|-------------------------|-----------------|---------------------|------------------------|---------------------------|
| Very Likely             | —               | —                   | Poor Generalization    | Data Risk                 |
| Likely                  | —               | —                   | Performance Drift Risk | Model Overfitting         |
| Unlikely                | —               | Explainability Risk | Ethical Risk           | Security Risk             |
| Very Unlikely           | —               | —                   | Monitoring Failure     | Regulatory Non-Compliance |





# Tools and Techniques used for Model Development

## Tools Used

- Python – Primary programming language for model development
- TensorFlow & Keras – For building and training CNN models
- Scikit-learn – For implementing classical ML algorithms and ensemble models
- OpenCV – For image preprocessing and resizing
- Flask – To create a simple web interface for model deployment
- Git & GitHub – For version control and collaboration

## Techniques Used

- Convolutional Neural Networks (CNN) for feature extraction
- Ensemble Learning (Majority Voting)
- MLOps lifecycle for deployment, monitoring, and maintenance





# Phase I : Machine Learning Model and its analysis

- Phase I focuses on building and evaluating the Machine Learning model for skin disease classification.
- A Convolutional Neural Network (CNN) is designed to automatically extract meaningful features from skin images.
- The CNN learns patterns related to color, texture, edges, and lesion shape, which are critical for skin disease identification.
- The trained CNN's penultimate layer is used as a feature extractor for further classification.



# Phase I : Machine Learning Model and its analysis

## Data Preprocessing

Original dataset contains skin disease images from multiple body parts

Five representative disease classes selected:

- Atopic Dermatitis
- Eczema
- Benign Keratosis-like Lesions (BKL)
- Tinea / Ringworm / Fungal Infections
- Seborrheic Keratoses & other Benign Tumors

Skin disease classes were randomly selected to ensure **visual diversity across anatomical regions**

Random sampling prevents bias toward specific body locations and **improves model generalization** under varied lighting and texture conditions

To address class imbalance, **1,250 images per class** were uniformly sampled

All images were resized to **224 × 224 pixels** and pixel values were normalized to the [0, 1] range

Standardization ensures consistent input representation and stable gradient updates during training

# Phase I : Machine Learning Model and its analysis

## Data Preprocessing

Data Augmentation was applied to improve robustness and reduce overfitting

Geometric transformations:

- Horizontal flipping
- Random rotation
- Zooming
- Width & height shifting
- Shear transformations

This simulates real-world variations (lighting, focus, texture noise)







# Phase I : Machine Learning Model and its analysis

## Feature Engineering

Features extracted using a **custom deep CNN**. CNN learns hierarchical and discriminative visual representations. This captures both **low-level and high-level image features**

Four convolutional blocks with increasing filters:

- $32 \rightarrow 64 \rightarrow 128 \rightarrow 256$

Each block includes **Batch Normalization** to stabilize learning, **Max Pooling** for spatial down-sampling, and **Dropout regularization** to prevent overfitting.

- Extracted deep features act as the selected feature set
- Encapsulate key visual cues:
  - Lesion shape
  - Texture patterns
  - Color variations

Intermediate feature vectors can be directly used for classification, or passed to traditional ML classifiers



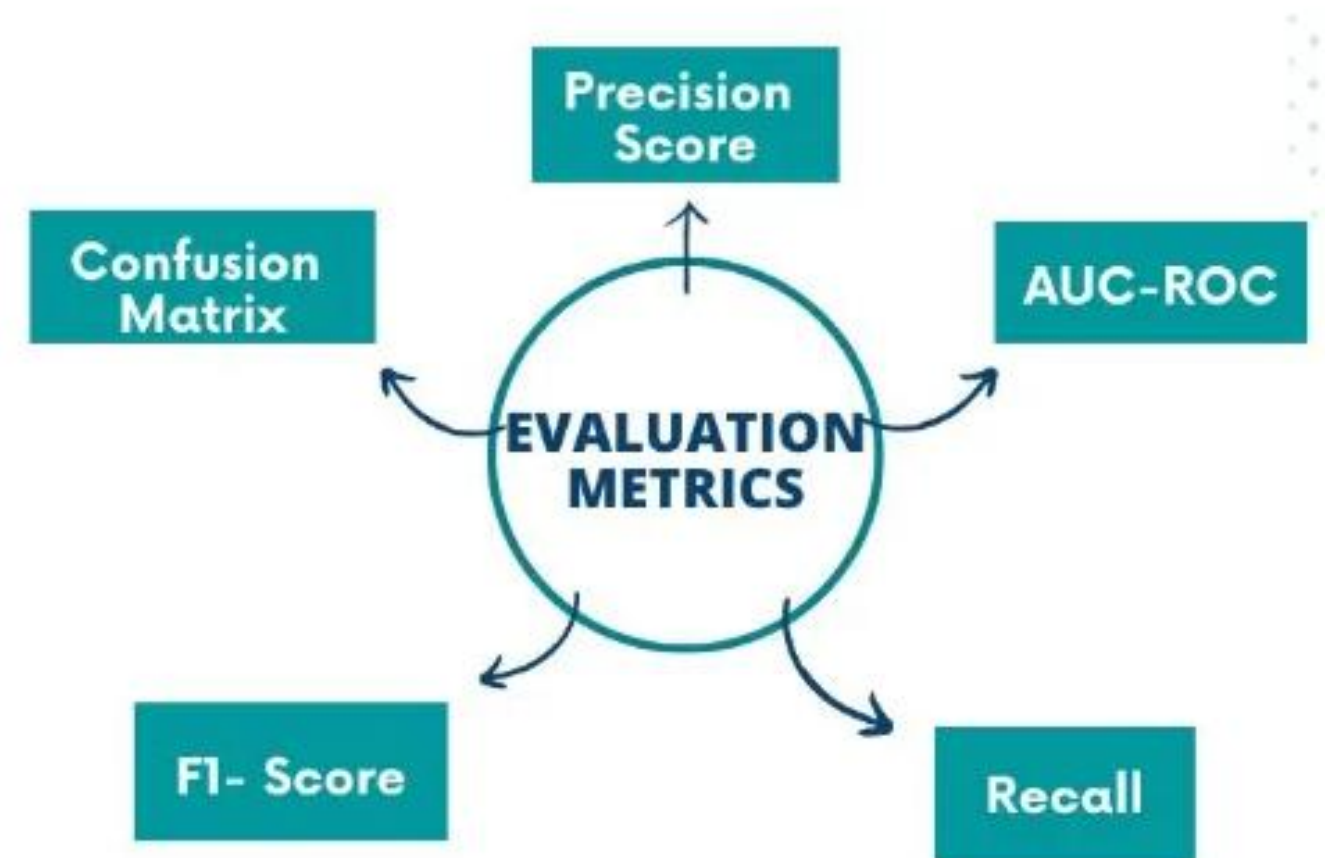


# List of algorithms chosen to build ML model

- **CNN-Based Feature Extraction** - A custom Convolutional Neural Network (CNN) is built and trained on the training set to learn discriminative visual features. Once trained, the CNN's penultimate layer is used as a feature extractor, transforming each image into a high-dimensional feature vector.
- **Ensemble-Based Classification** - The extracted feature vectors were then passed to an ensemble of classical machine learning models for final prediction. The ensemble consists of:
  - Random Forest (RF)
  - Logistic Regression (LR)
  - Support Vector Machine
  - K-Nearest Neighbors

# Metrics used to evaluate the ML model

- **Accuracy:** Measures the overall percentage of correctly classified skin disease images.
- **Precision:** Indicates how many of the predicted positive cases are actually correct, helping reduce false positives.
- **Recall(Sensitivity):** Measures the model's ability to correctly identify actual disease cases, which is critical in medical diagnosis.
- **F1-Score:** Harmonic mean of precision and recall, providing a balanced evaluation for multi-class classification.
- **Area Under Curve (AUC):** Evaluates the model's ability to distinguish between different disease classes across thresholds.



# Evaluation of ML model

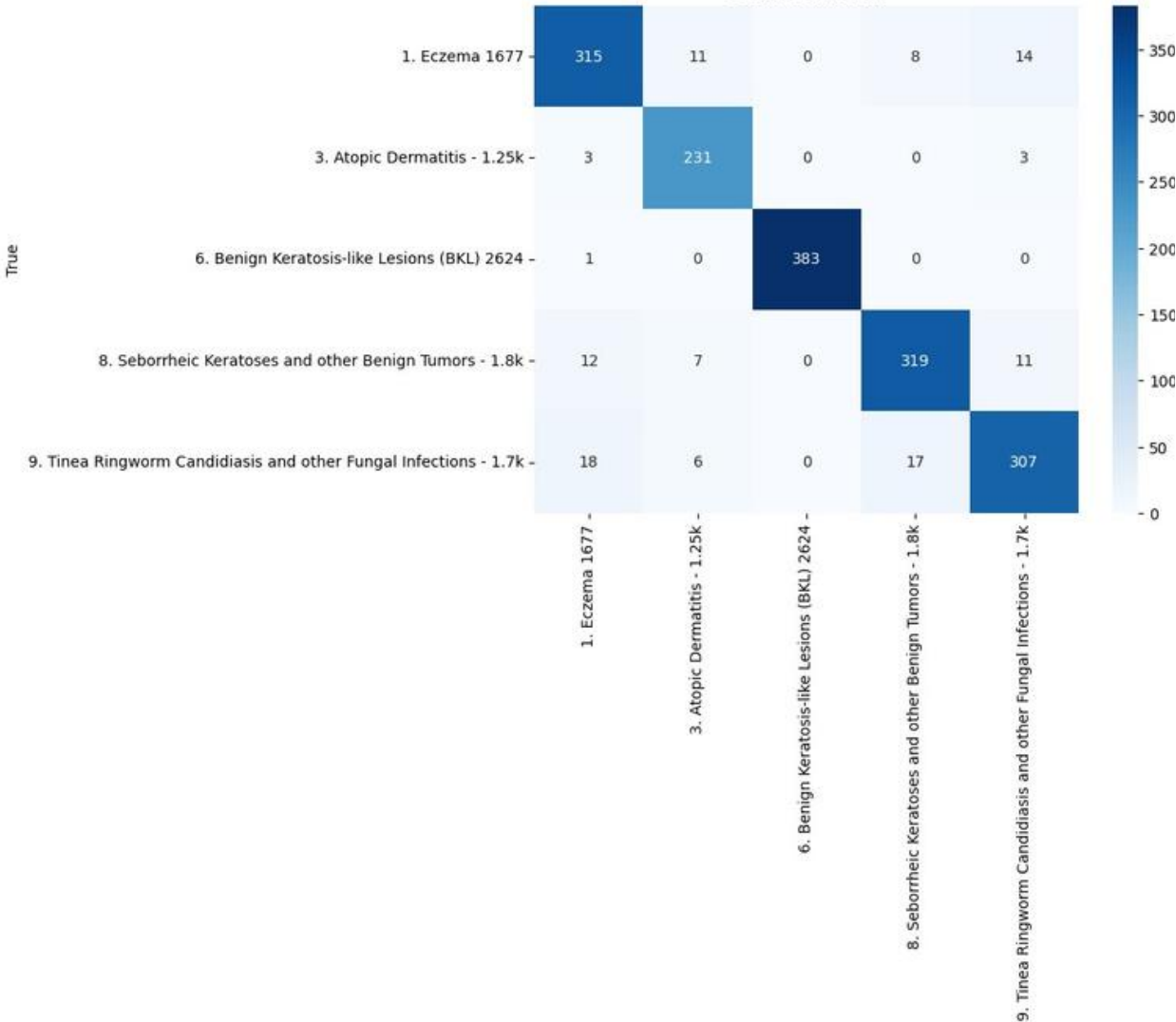
The proposed hybrid model combines **CNN-based feature extraction** with an **ensemble of traditional machine learning classifiers**, Logistic Regression, Random Forest, K-Nearest Neighbors (KNN), and Support Vector Machine (SVM) using a **majority voting mechanism**.

Classification Report:

|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| 0            | 0.90      | 0.91   | 0.90     | 348     |
| 1            | 0.91      | 0.97   | 0.94     | 237     |
| 2            | 1.00      | 1.00   | 1.00     | 384     |
| 3            | 0.93      | 0.91   | 0.92     | 349     |
| 4            | 0.92      | 0.88   | 0.90     | 348     |
| accuracy     |           |        | 0.93     | 1666    |
| macro avg    | 0.93      | 0.93   | 0.93     | 1666    |
| weighted avg | 0.93      | 0.93   | 0.93     | 1666    |

The ensemble was evaluated on a **balanced test set of 1,666 images**, representing five major skin conditions with roughly **equal support per class**.

The ensemble model achieved the best overall performance, with a testing accuracy of **93.34%**, precision, recall, and F1-score all at 93.3%, and a high AUC score of 0.988, indicating strong discriminative power.



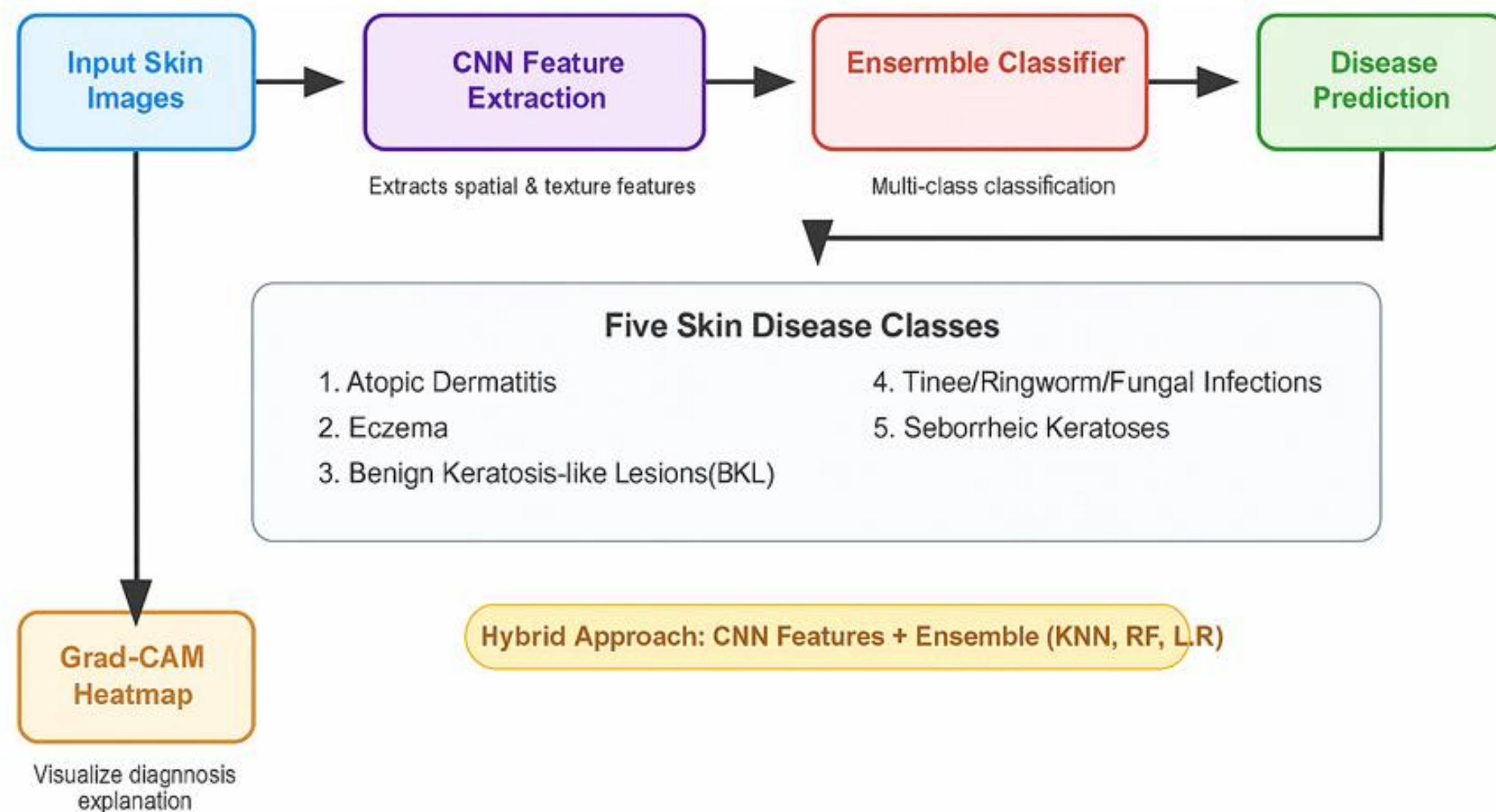


# Evaluation of ML model

| Model                  | Accuracy | Precision | Recall | F1-Score |
|------------------------|----------|-----------|--------|----------|
| Logistic Regression    | 92.49%   | 92.51%    | 92.49% | 92.49%   |
| Random Forest          | 93.15%   | 93.24%    | 93.15% | 93.15%   |
| K-Nearest Neighbors    | 92.01%   | 92.22%    | 92.01% | 92.02%   |
| Support Vector Machine | 90.69%   | 90.78%    | 90.69% | 90.69%   |
| Ensemble               | 93.34%   | 93.35%    | 93.33% | 93.32%   |



# Identification of Best Model



- Based on the evaluation results, the Ensemble Model was identified as the best-performing model.
- The ensemble combines predictions from:
  - Logistic Regression
  - Random Forest
  - K-Nearest Neighbors
  - Support Vector Machine
- Majority voting was used to determine the final class label.

This model showed improved generalization and reduced misclassification compared to individual models.





## Phase II : Building MLOPs

- Phase II focuses on applying MLOps principles to manage the complete lifecycle of the machine learning model.
- MLOps ensures that the ML system is:
  - Reproducible
  - Scalable
  - Maintainable
  - Deployable in real-world environments
- The MLOps pipeline covers:
  - Data Management
  - Experiment Tracking and Versioning
  - Automation and Orchestration
  - Model Deployment and Serving
  - Monitoring, Governance, and Collaboration



# Phase II : Building

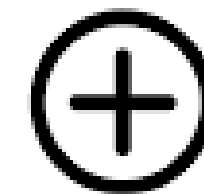
|   | Component                                 | MLOPs Tools              |
|---|-------------------------------------------|--------------------------|
| 1 | Data Management                           | DVC + Git                |
| 2 | Experiment Tracking and Version Control   | ZenML+Weights and Biases |
| 3 | Automation and Pipeline Orchestration     | ZenML + MinIO            |
| 4 | Model Deployment & Serving                | FastAPI + Docker         |
| 5 | Monitoring & Governance and collaboration | Evidently AI             |

# Data Management

Ensuring data consistency, versioning, and reproducibility across ML experiments.

Tool: DVC (Data Version Control) + Git

- DVC stores large datasets and trained models in remote storage (S3, GCP, etc.)
- Git tracks lightweight .dvc metadata files instead of raw data
- Enables data lineage and experiment reproducibility



Key Features:

- Large file handling – Keeps data separate from source code
- Pipeline caching – Avoids re-running unchanged stages
- Collaboration – Teams share dataset versions using Git
- Integration – Works with ML frameworks and cloud platforms

Use in our project:

- Version skin image datasets
- Tracking preprocessing and training inputs
- Ensure consistent data across training and inference

# Experiment Tracking and Version Control

Experiment tracking is used to manage, compare, and reproduce machine learning experiments across multiple development iterations.

Tool: ZenML

How it works:

- ZenML provides a structured framework for building and tracking end-to-end machine learning pipelines.
- Each pipeline run automatically records experiment metadata, including parameters, metrics, dataset versions, and model artifacts.
- It ensures that every experiment is reproducible by linking pipeline executions with specific code states and data versions.

Key Features:

- Pipeline-based tracking – Ensures reproducible end-to-end workflows
- Artifact versioning – Manages models, features, and outputs
- Metadata logging – Records metrics and configurations per run
- Reproducibility – Allows experiments to be rerun with identical settings

Use in our project:

- Track accuracy, loss, and other metrics across runs
- Compare different architectures
- Log confusion matrices and classification reports

# Automation and Pipeline Orchestration

Automating end-to-end ML workflows from data ingestion to model training.

Tool: ZenML + MinIO (Artifact Store)

- ZenML defines ML pipelines as code using steps and artifacts
- MinIO serves as the S3-compatible artifact store for ZenML
- Stores model versions, pipeline outputs, and metadata
- Integrates with DVC and deployment tools

Key Features:

- Modularity – Reusable pipeline components
- Artifact tracking – MinIO stores all pipeline outputs versioned
- Reproducibility – Consistent environments with containers
- Orchestration – Schedule and automate pipeline runs



Use in our project:

- Orchestrate data ingestion → preprocessing → training → evaluation
- Store trained models, preprocessing objects, and evaluation reports in MinIO
- Ensure pipeline consistency across development and production





# Automation and Pipeline Orchestration

## Tool: MinIO

- MinIO provides object storage compatible with Amazon S3 API
- Integrated with ZenML as the artifact store
- Stores: trained models, datasets, pipeline outputs, metadata

## Benefits:

- High performance – Optimized for ML workloads
- Scalable – Handles large volumes of artifacts
- On-premise or cloud – Deploy anywhere
- Secure – Encryption and access control

## Use in our project:

- Store all pipeline outputs (models, scalars, encoders)
- Version models automatically with ZenML + MinIO
- Retrieve artifacts for deployment or further analysis



# Model Deployment & Serving

Deploying trained models as scalable, production-ready APIs.

Tool: FastAPI + Docker

- FastAPI creates high-performance REST APIs with automatic documentation
- Docker containers package the model, dependencies, and environment
- Can be deployed on cloud platforms or on-premise servers

Key Features:

- Low latency – Asynchronous support for fast inference
- Scalability – Containerized deployment supports scaling (K8s, ECS)
- Security – Input validation, authentication, and logging

Use in our project:

- Deploying **Zenml** server
- Deploy skin disease classification **model as an API**
- Allow **image upload** and **return prediction** with confidence score
- Enable integration with mobile/web applications
- Fetch latest model version from **MinIO** via **ZenML**



# Monitoring & Governance and collaboration

Ensuring model performance, data drift detection, and collaborative governance.

Tool: Evidently AI

- Evidently AI analyzes data drift, model performance, and bias over time
- GitHub manages code versions, CI/CD, and collaboration
- Combined for audit trails and governance



Key Features:

- Drift detection – Monitors feature distributions and model decay
- Interactive reports – Visual dashboards for stakeholders
- Version control – GitHub for code, model, and pipeline versioning
- CI/CD integration – Automated testing and deployment checks

Use in our project:

- Monitor real-time predictions vs. training data
- Detect bias in skin tone representation
- Generate compliance reports for clinical use
- Maintain audit trail with GitHub commits and Evidently logs





# Conclusion

- Developed a hybrid skin disease classification system using CNN-based feature extraction and an ensemble of traditional ML classifiers.
- Individual classifiers were evaluated, and a majority voting ensemble was adopted to achieve improved and stable performance.
- Implemented MLOps practices using ZenML for experiment tracking and pipeline management, with DVC and Git for data and code versioning.
- Used MinIO for artifact storage, enabling reproducibility and systematic model management.
- Integrated Evidently AI for feature-level drift detection to monitor data distribution changes and support reliable model maintenance.



**RV College of  
Engineering®**

Mysore Road, RV Vidyaniketan f  
Bengaluru - 560059, Karnataka,

*Go, change the  
world*

# Thank You

Department of AI and ML